

VISÃO COMPUTACIONAL APLICADA

# Leitor de gabarito

## Da foto ao placar, com conferência visível

---

8 questões · 4 alternativas · Streamlit · OpenCV · Pillow

# O problema

---

Uma folha fotografada não chega ao computador em condições ideais:

- perspectiva, rotação e escala variam a cada foto;
- sombra e iluminação desigual mudam o brilho da página;
- uma questão pode ter duas marcas, uma marca fraca ou nenhuma marca;
- o nome manuscrito exige uma etapa diferente da correção objetiva.

**Objetivo:** separar geometria, leitura e regra de negócio para que cada parte seja testável.

# O fluxo completo

---

```
foto
  ↓
4 marcadores ArUco
  ↓
homografia → imagem canônica 1000 × 1483
  ↓
32 medições de preenchimento
  ↓
estado de cada questão
  ↓
placar + tabela + imagem de conferência
```

O nome segue um caminho paralelo e opcional:

```
imagem canônica → recorte → Gemini → texto editável
```

# A folha é a fonte da geometria

---

O módulo `layout.py` é compartilhado pelo gerador e pelo leitor.

| Elemento                 | Valor                                    |
|--------------------------|------------------------------------------|
| Região canônica          | 1000 × 1483 px                           |
| Questões                 | 8                                        |
| Alternativas por questão | 4 (A–D)                                  |
| Bolinhas medidas         | 32                                       |
| Marcadores               | ArUco <code>DICT_4X4_50</code> , IDs 0–3 |

Isso evita que a folha seja desenhada com uma geometria e lida com outra.

# 1. Alinhamento por ArUco

---

Os quatro marcadores identificam os cantos e também revelam a orientação.

1. Detectar os IDs 0, 1, 2 e 3.
2. Pegar o canto externo de cada marcador.
3. Calcular a homografia para o retângulo canônico.
4. Reprojetar a foto em tons de cinza.

O resultado corrige perspectiva, rotação e escala de uma vez.

**Sem os quatro marcadores, não há placar parcial.**

## 2. Medir, não detectar círculos

Depois do alinhamento, os centros das bolinhas já são conhecidos pelo layout.

anel: papel local



← 80% do raio

Para cada bolinha:

```
preenchimento = 1 - média(disco) / média(anel)
```

O papel ao redor funciona como referência local. Assim, uma sombra não transforma automaticamente todas as bolinhas em marcas.

# Dois sinais, uma decisão cautelosa

---

| Preenchimento relativo | Leitura |
|------------------------|---------|
| $< 0,20$               | vazia   |
| $> 0,50$               | marcada |
| entre os dois          | ambígua |

O `adaptiveThreshold` roda como conferência. Quando os sinais discordam, a leitura fica **ambígua** e a questão pode ir para `REVISAR`.

O threshold adaptativo nunca substitui a medida relativa local.

### 3. Regras de correção

---

O módulo `corrigir.py` não conhece imagens. Recebe somente as leituras e a chave.

| Condição                     | Estado     | Pontua?          |
|------------------------------|------------|------------------|
| Uma marcada, sem ambiguidade | RESPONDIDA | se igual à chave |
| Duas ou mais marcadas        | ANULADA    | não              |
| Nenhuma marcada              | EM BRANCO  | não              |
| Qualquer ambiguidade         | REVISAR    | não              |

**Anulada tem prioridade sobre revisar:** uma questão com duas marcas definitivas não pode virar acerto por causa de uma terceira leitura incerta.

## 4. A imagem de conferência

---

O usuário não recebe apenas **6/8**. A aplicação desenha o motivo:

- **verde:** marcada e correta;
- **vermelho:** marcada e errada;
- **amarelo:** leitura ambígua;
- **cinza:** questão anulada.

Essa camada é simultaneamente interface, evidência e ferramenta de depuração.

# O nome é opcional

---

1. Recortar `NOME_RECT` da imagem canônica.
2. Enviar o recorte ao `gemini-2.5-flash` se existir `GEMINI_API_KEY`.
3. Mostrar o texto em um campo editável.

Sem chave, sem rede, com erro da API ou com resposta ilegível: o recorte continua visível e o placar continua funcionando.

O nome pode ser assistido por IA; a nota não depende dela.

# A interface em uma tela

---

## Barra lateral

- oito seletores da chave de respostas;
- download da folha em PDF;
- instruções de enquadramento e impressão.

## Área principal

- upload ou câmera;
- conferência anotada;
- recorte e campo do nome;
- placar **x/8** e tabela das oito questões.

# Testes com verdade conhecida

---

O gerador sintético desenha a folha pelo mesmo `layout.py`, preenche respostas conhecidas e simula uma foto de celular:

| Cenário                                | Cobertura              |
|----------------------------------------|------------------------|
| 200 folhas aleatórias                  | integração do pipeline |
| perspectiva, rotação e escala          | alinhamento            |
| gradiente, sombra, desfoque e JPEG     | robustez da leitura    |
| cabeça para baixo e escalas diferentes | orientação e geometria |
| marca dupla, vazia e ambígua           | regras de correção     |

**Resultado validado:** `286 passed` com `uv run pytest -q`.

# Demonstração sugerida

---

```
uv run python scripts/gerar_demos.py  
uv run streamlit run app.py
```

1. Enviar `02_anulada_branco_entrada.jpg` : acertos, anulação e vazio.
2. Enviar `03_erro_revisar_entrada.jpg` : erro e revisão na imagem.
3. Alterar uma chave na barra lateral: o placar muda imediatamente.
4. Testar `05_cabeca_baixo_entrada.jpg` : a orientação é recuperada pelos IDs.

# O que ainda não está resolvido

---

- Ainda não há validação com papel real.
- Reflexos, dobras, amassados e desfoque de movimento podem divergir do sintético.
- Os quatro marcadores precisam estar inteiros na foto.
- Uma marca em ☐ ou ☒ pode parecer vazia.
- A transcrição manuscrita não é verificada automaticamente.

O sistema prefere pedir uma nova foto a inventar um resultado.

# Em uma frase

Marcadores resolvem a geometria; medidas locais resolvem a iluminação; regras puras resolvem a nota; a conferência mantém a decisão auditável.

Perguntas?